

Documentação do Projeto: Consultas Médicas API

Capa

Item	Detalhe
Disciplina	Java
Curso	FIAP
Nome da Solução	Consultas Médicas API
Nome da Equipe	TeleCare
Nome dos Integrantes	Gabriel Maciel Alves de Oliveira (RM 562795), Thomas Mineu Fontes (RM 562254), Matheus Pereira Molina (RM 563399)
Repositório GitHub	Gabriel-Maciel06/CH-JAVA

Objetivo e Escopo do Projeto

Descrição Objetiva e Concisa da Solução Proposta

A **Consultas Médicas API** é uma API RESTful desenvolvida em Java com Quarkus, focada no gerenciamento de agendamentos de consultas médicas. Seu principal diferencial é a implementação de um **Sistema de Cálculo de Risco de Absenteísmo** (falta do paciente), que utiliza regras de negócio para classificar consultas como de “Alto Risco”, permitindo ações proativas para confirmar a presença do paciente. O projeto segue a arquitetura em camadas (TO/DAO/BO/Resource) e utiliza JDBC puro para acesso ao banco de dados Oracle.

Breve Descrição das Principais Funcionalidades da Solução

O sistema oferece funcionalidades completas de CRUD (Create, Read, Update, Delete) para as principais entidades do domínio: Pacientes, Médicos e Consultas. As funcionalidades implementadas incluem:

- Gerenciamento de Pacientes e Médicos:** Cadastro e manutenção de dados básicos.

2. **Agendamento de Consultas:** Criação de consultas, com validações e cálculo automático do risco de absenteísmo.
3. **Cálculo de Risco de Absenteísmo:** Um serviço de negócio que calcula um índice de risco (0 a 100) baseado em:
 - Total de faltas do paciente (20 pontos por falta).
 - Tipo de consulta (15 pontos para consultas ONLINE).
 - Ausência de canal de comunicação preferencial (10 pontos).
 - Consultas com risco ≥ 60 são classificadas como de Alto Risco.
4. **Confirmação e Recaptura:** Endpoints específicos para confirmar consultas e para recalculer o risco de absenteísmo, refletindo o Domain Driven Design (DDD) com ações explícitas.
5. **Notificações:** Registro de notificações enviadas aos pacientes.
6. **Teste de Dispositivo:** Registro de testes de câmera, microfone e conexão para consultas online.

Tabela de Endpoints (API Restful)

A API é estruturada em torno de recursos RESTful, com verbos HTTP e códigos de status apropriados.

Recurso: `/api/pacientes`

Verbo HTTP	URI (Caminho do Recurso)	Descrição	Códigos de Status Esperados
GET	<code>/api/pacientes</code>	Lista todos os pacientes.	200 (OK)
GET	<code>/api/pacientes/{id}</code>	Busca paciente por ID.	200 (OK), 404 (Not Found)
POST	<code>/api/pacientes</code>	Cria novo paciente.	201 (Created), 400 (Bad Request)
PUT	<code>/api/pacientes/{id}</code>	Atualiza paciente.	200 (OK), 400 (Bad Request)
DELETE	<code>/api/pacientes/{id}</code>	Remove paciente.	204 (No Content), 500 (Internal Server Error)

Recurso: /api/consultas

Verbo HTTP	URI (Caminho do Recurso)	Descrição	Códigos de Status Esperados
GET	/api/consultas	Lista todas as consultas.	200 (OK)
GET	/api/consultas/{id}	Busca consulta por ID.	200 (OK), 404 (Not Found)
GET	/api/consultas/alto-risco	Lista consultas com risco >= 60.	200 (OK)
POST	/api/consultas	Cria nova consulta (calcula risco automaticamente).	201 (Created), 400 (Bad Request)
POST	/api/consultas/{id}/confirmar	Confirma uma consulta.	200 (OK), 400 (Bad Request)
POST	/api/consultas/{id}/recapturar	Recalcula o risco de absenteísmo.	200 (OK), 400 (Bad Request)
PUT	/api/consultas/{id}	Atualiza consulta.	200 (OK), 400 (Bad Request)
DELETE	/api/consultas/{id}	Remove consulta.	204 (No Content), 500 (Internal Server Error)

Outros Recursos

Recurso	Verbo HTTP	URI (Caminho do Recurso)	Descrição
Médicos	GET , POST , PUT , DELETE	/api/medicos e /api/medicos/{id}	CRUD completo para Médicos.
Notificações	GET , POST , PUT , DELETE	/api/notificacoes e /api/notificacoes/{id}	CRUD completo para Notificações.
Teste de Dispositivo	GET , POST , PUT , DELETE	/api/device-check e /api/device-check/{id}	CRUD completo para Testes de Dispositivo.

Protótipo - Exemplos de Interação com a API

Demonstração Visual: O funcionamento da API e a interação com o sistema podem ser visualizados na demonstração em vídeo: [Demonstração do Projeto](#)

Como o projeto é uma API RESTful, o protótipo é apresentado através de exemplos de requisições `CURL` que demonstram a interação com os principais endpoints e funcionalidades.

1. Criar um Paciente (POST)

Descrição: Cria um novo paciente no sistema. **Requisição:**

```
curl -X POST http://localhost:8080/api/pacientes \  
-H "Content-Type: application/json" \  
-d '{  
  "nmPaciente": "João da Silva",  
  "dsTelefone": "11999887766",  
  "dsCanalPref": "WHATSAPP",  
  "nmCuidador": "Maria da Silva",  
  "dsTelCuidador": "11999887767"  
}'
```

2. Criar uma Consulta (POST)

Descrição: Agenda uma nova consulta. O sistema calcula o `vlRiscoAbs` automaticamente. **Requisição:**

```
curl -X POST http://localhost:8080/api/consultas \  
-H "Content-Type: application/json" \  
-d '{  
  "idPaciente": 1,  
  "idMedico": 1,  
  "dtConsulta": "2025-11-20T10:00:00",  
  "dsTipo": "ONLINE"  
}'
```

3. Listar Consultas de Alto Risco (GET)

Descrição: Demonstra a funcionalidade de negócio, listando apenas as consultas cujo risco de absenteísmo é igual ou superior a 60. **Requisição:**

```
curl http://localhost:8080/api/consultas/alto-risco
```

4. Confirmar uma Consulta (POST)

Descrição: Ação explícita para confirmar a presença do paciente, alterando o status da consulta. **Requisição:**

```
curl -X POST http://localhost:8080/api/consultas/1/confirmar
```

Modelo de Entidade-Relacionamento (MER)

O modelo de dados é composto por cinco entidades principais, refletindo a estrutura das classes *TO e a arquitetura do banco de dados.

Entidades e Atributos

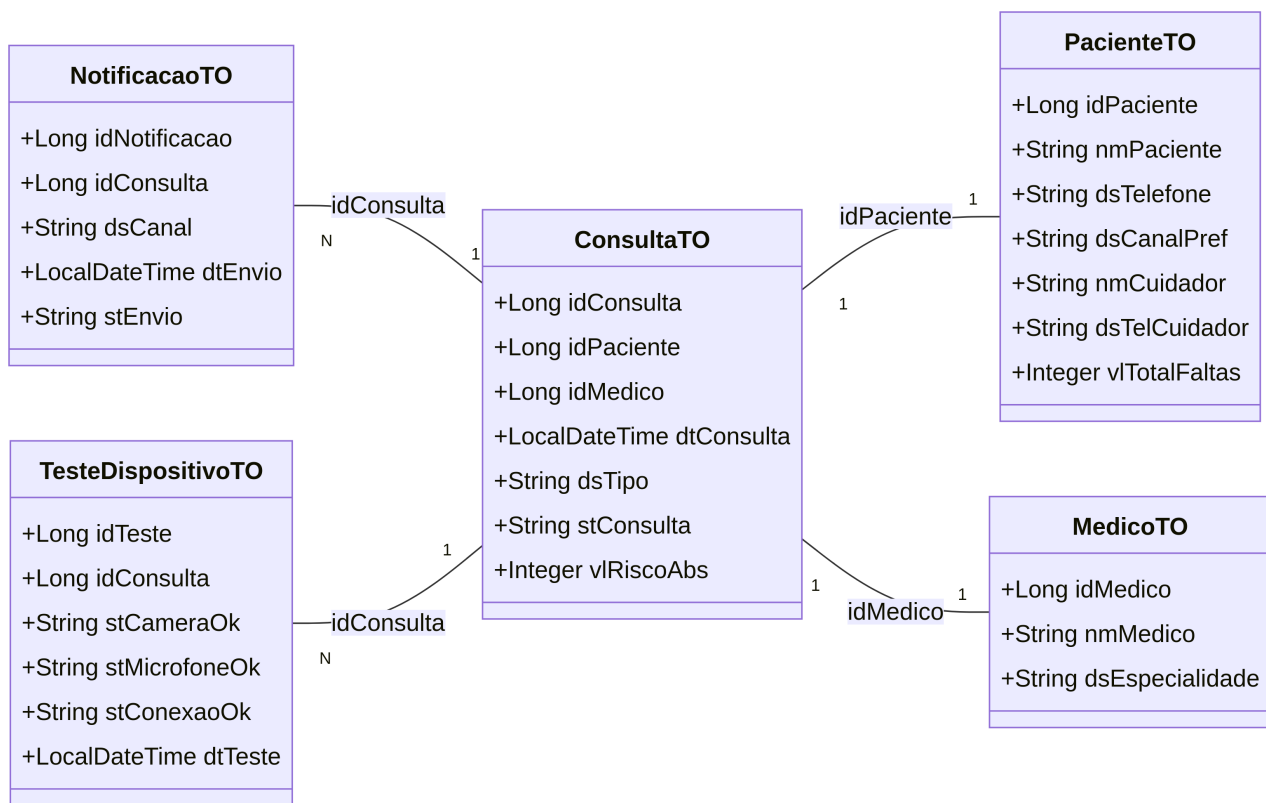
Entidade	Chave Primária	Atributos
T_PACIENTE	ID_PACIENTE	NM_PACIENTE , DS_TELEFONE , DS_CANAL_PREF , NM_CUIDADOR , DS_TEL_CUIDADOR , VL_TOTAL_FALTAS
T_MEDICO	ID_MEDICO	NM_MEDICO , DS_ESPECIALIDADE
T_CONSULTA	ID_CONSULTA	ID_PACIENTE (FK), ID_MEDICO (FK), DT_CONSULTA , DS_TIPO , ST_CONSULTA , VL_RISCO_ABS
T_NOTIFICACAO	ID_NOTIFICACAO	ID_CONSULTA (FK), DS_CANAL , DT_ENVIO , ST_ENVIO
T_TESTE_DISPOSITIVO	ID_TESTE	ID_CONSULTA (FK), ST_CAMERA_OK , ST_MICROFONE_OK , ST_CONEXAO_OK , DT_TESTE

Relacionamentos

Relacionamento	Entidade A	Cardinalidade	Entidade B	Cardinalidade	Descrição
Agendamento	T_CONSULTA	N	T_PACIENTE	1	Uma consulta é agendada para um paciente.
Atendimento	T_CONSULTA	N	T_MEDICO	1	Uma consulta é realizada por um médico.
Notificação	T_NOTIFICACAO	N	T_CONSULTA	1	Uma consulta pode gerar várias notificações.
Teste	T_TESTE_DISPOSITIVO	N	T_CONSULTA	1	Uma consulta pode ter vários testes de dispositivo.

Diagrama de Classes Atualizado

O diagrama de classes reflete a estrutura das classes de Transfer Object (***TO**) e seus relacionamentos, que representam o modelo de domínio da aplicação.



classDiagram direction LR class PacienteTO {

```

+Long idPaciente
+String nmPaciente
+String dsTelefone
+String dsCanalPref
+String nmCuidador
+String dsTelCuidador
+Integer vlTotalFaltas

```

}

class MedicoTO {

```

+Long idMedico
+String nmMedico
+String dsEspecialidade

```

}

class ConsultaTO {

```
+Long idConsulta  
+Long idPaciente  
+Long idMedico  
+LocalDateTime dtConsulta  
+String dsTipo  
+String stConsulta  
+Integer vlRiscoAbs
```

```
}
```

```
class NotificacaoTO {
```

```
+Long idNotificacao  
+Long idConsulta  
+String dsCanal  
+LocalDateTime dtEnvio  
+String stEnvio
```

```
}
```

```
class TesteDispositivoTO {
```

```
+Long idTeste  
+Long idConsulta  
+String stCameraOk  
+String stMicrofoneOk  
+String stConexaoOk  
+LocalDateTime dtTeste
```

```
}
```

ConsultaTO “1” – “1” PacienteTO : idPaciente ConsultaTO “1” – “1” MedicoTO : idMedico
NotificacaoTO “N” – “1” ConsultaTO : idConsulta TesteDispositivoTO “N” – “1” ConsultaTO :
idConsulta “`

Detalhes da Implementação (Requisitos Adicionais)

Camada Model (TO)

As classes *TO (Transfer Object) representam o modelo de dados e incluem validações de negócio utilizando **Jakarta Bean Validation** (`@NotNull`, `@NotBlank`, `@Pattern`), garantindo a integridade dos dados antes do processamento.

Camada DAO e Service (BO)

- **DAO (Data Access Object):** Responsável pela comunicação direta com o banco de dados Oracle, utilizando **JDBC puro**.
- **BO (Business Object):** Camada de serviço que implementa as regras de negócio, como o **Cálculo de Risco de Absenteísmo** e a lógica de CRUD. O CRUD está completamente funcional, e as regras de negócio estão adequadamente encapsuladas.

API Restful

A implementação segue os princípios REST:

- **Recursos:** Endpoints bem definidos (`/api/consultas` , `/api/pacientes` , etc.).
- **Verbos HTTP:** Uso correto de `GET` , `POST` , `PUT` , `DELETE` .
- **Status de Resposta:** Retorno de códigos de status apropriados (200, 201, 204, 400, 404, 500).

Boas Práticas e Padrões de Projeto

Padrão de Projeto	Local de Aplicação	Descrição
DAO	Classes <code>*DAO</code>	Separação da lógica de acesso a dados.
DTO/TO	Classes <code>*TO</code>	Transferência de dados entre camadas.
Singleton	<code>ConnectionFactory</code>	Garante uma única instância para o gerenciamento de conexões.
Factory	<code>ConnectionFactory</code>	Criação de objetos de conexão de forma centralizada.
Arquitetura em Camadas	<code>TO/DAO/BO/Resource</code>	Organização clara e modular do código.

O código segue regras de nomenclatura e organização, e o tratamento de exceções é realizado nas camadas `Resource` e `BO` , retornando respostas HTTP adequadas ao cliente.